How the organisation has to work

The operating model made operational. What every product owns, how a change gets approved, how thirty-three bounded contexts become nine teams, how a merchant estate moves without pretending the migration is free, and what the first ninety days actually contain.

Document	ST-001 · v1.0
Date	27 September 2026
Consolidates	Governance and operating model, PV-2027 Parts III–IV, the delivery backlog
Companion	VS-001 · the vision it executes
Implements	EA-001 v1.1 · PD-001 · PD-002
Status	Canonical

Contents

PART I — THE OPERATING MODEL

01 What this document is for

02 The seven obligations

03 Thirty-three contexts, nine teams

04 The product team charter

05 Entitle by pattern, restrict by policy

PART II — GOVERNANCE

06 One change model

07 Roles and segregation

08 Explainability as an operating requirement

09 The interlock with bank governance

10 Failure modes to design against

PART III — GETTING THERE

11 The build sequence

12 Six migration stages

13 The first ninety days

14 What it costs

PART

I — The operating model

01 What this document is for

02 The seven obligations

03 Thirty-three contexts, nine teams

04 The product team charter

05 Entitle by pattern, restrict by policy

01

What this document is for

> VS-001 says where we are going. EA-001 and PD-001 say what gets built. This says who owns what, how a change gets approved, and in what order any of it happens.

THE PROBLEM, STATED PLAINLY

We are building a platform whose whole proposition is that behaviour changes without a release: strategies, policies, plans and preferences are all deployable artefacts. That is the point, and it is also the risk.

Modularity does not remove risk. It converts release risk into configuration risk. A release has a change ticket, a reviewer, a deployment window and a rollback. A configuration change, by default, has none of those — and there are far more of them. Everything in Part II exists to stop that conversion being a downgrade.

The second problem is organisational and less obvious. An architecture of thirty-three bounded contexts is the right granularity for versioning and ownership of artefacts. It is the wrong granularity for headcount, and an organisation that tries to staff it directly produces thirty-three under-resourced teams that become shared-service bottlenecks and recreate the monolith socially. Section 03 is the answer.

02

The seven obligations

> Every product owns its whole life, from the moment a client turns it on to the moment something goes wrong at three in the morning. Seven obligations, and a product that meets six is a feature somebody else has to operate.

THE SEVEN OBLIGATIONS · WHAT EVERY PRODUCT OWNS END TO END

SETUP
how a client turns it on, without a ticket

INTEGRATION
how a developer reaches it, documented

EXECUTION
how it runs, with a stated SLO

MONETIZATION
what it costs, on a rate card

METERING
who used how much of it, measured

REPORTING
what the client can see about it

SERVICING
what happens when it goes wrong, and who

A PRODUCT THAT MEETS SIX OF SEVEN IS A FEATURE SOMEBODY ELSE HAS TO OPERATE

The obligations are the anti-silo mechanism: they make the cost of a half-built product land on the team that built it.

— Metering and servicing are the two that get dropped, and they are the two that decide whether the platform can be sold as a service rather than operated as a favour.

OBLIGATION	WHAT IT MEANS CONCRETELY	THE TEST
Setup	A client can turn it on through the boarding and servicing experience, without a ticket and without an internal handoff	Can a solutions engineer enable it in a demo?
Integration	A developer can reach it through the documented API, with a sandbox that produces its failure modes as well as its successes	Can somebody integrate it without talking to us?
Execution	It runs, with a stated SLO and an owner on call	Is there a named person when it breaks at 3am?
Monetization	It has a price on a rate card, not a negotiation	Can sales quote it without asking?
Metering	Usage is measured per client, in the units the price is in	Can finance invoice it?
Reporting	The client can see what it did for them	Can the merchant answer their own question without a support ticket?
Servicing	There is a defined path when it goes wrong, and the first-line team can follow it	Does an incident have a runbook, or a phone number?

THE TWO THAT GET DROPPED, AND WHY IT MATTERS

Metering and servicing. They are the last two built and the first two cut, because neither is visible in a demo.

They are also exactly the two that decide whether the platform can be *sold as a service* rather than *operated as a favour*. A shared service without metering cannot be charged for, so it gets funded centrally, so it gets deprioritised against things that can be charged for. A shared service without servicing becomes a set of escalations to the team that built it, which is how a platform team stops building. [J]

The obligations are the anti-silo mechanism. They work by making the cost of a half-built product land on the team that built it, rather than on the operations, support and finance functions downstream of it — which is where that cost currently lands, invisibly.

03

Thirty-three contexts, nine teams

> Bounded contexts are a code and artefact property. Product groups are a headcount and accountability property. Conflating them is how a domain model becomes an org chart nobody can staff.

PRODUCT GROUP	OWNS	ROUGHLY
Commerce platform	The resource model, the API surface, the developer experience, industry contexts	The front door
Decision platform	Decision points, strategies, flows, experimentation, the learning layer	PD-002 in its entirety
Credential platform	Vault, tokenisation, instrument intelligence, the Forward API, account updater	The gate-2 asset
Authentication and trust	Authentication, risk, screening, fraud, policy	The constraint side
Processing platform	Orchestration, execution, adapters, rails, recovery, message intelligence	The engine
Money movement	Settlement, ledger, funding, treasury, currency, disputes	The correctness side — and a different change velocity
Client platform	Boarding and servicing orchestration, entitlement, the product and solution catalogue, reporting	Where the seven obligations are actually met
Reliability and security	The cryptographic zones, tenant isolation, platform reliability, the control plane's own integrity	The things nobody sells
Governance and data	The data platform, metering, model risk, the change model itself	The part that makes the rest auditable

WHY MONEY MOVEMENT IS A SEPARATE GROUP AND NOT A SEPARATE PLATFORM

A proposal to split the architecture into two platforms — decision and money movement — was declined, and the reasoning is worth keeping because it will be re-proposed.

The diagnosis was right: correctness requirements genuinely differ, and money movement should not inherit the change velocity of decisioning. **But that is an organisational and change-governance boundary, not an architectural one** — and drawing it as two platforms would duplicate the event fabric and the resource model, which is the one thing both of them need to share.

So the boundary exists here, in the group structure and in the change tiers, rather than in the architecture diagram. A right diagnosis landing in the wrong artefact is a common failure and this is the cleanest example of it in the corpus. [A]

04

The product team charter

Every product group publishes one. It is a page, not a document, and it answers six questions that are otherwise settled by whoever escalates loudest.

THE CHARTER STATES	WHY IT HAS TO BE WRITTEN DOWN
What this product is, in one sentence a merchant would recognise	If the sentence needs internal vocabulary, the product has no external existence and should be a capability inside another one
Which of the seven obligations it currently meets, honestly	The gap is the roadmap. A charter claiming all seven on day one is not a charter
Who it sells to, and which buyer segment leads	Four segments buy for different reasons; a product that claims all four has not chosen
What it consumes from other groups, and at what freshness	This is where the real coupling lives, and it is invisible until something goes stale
What it will refuse to build	The hardest line and the most useful one. A product with no declared edges absorbs every adjacent request
Its decision points, if it has any	A product that owns a decision point inherits the whole of PD-002: registration, fallbacks, propensity, guardrails and an expiry

Charters are reviewed annually and whenever a boundary moves. A charter that has not changed in two years is either a mature product or an unexamined one, and the review is how to tell.

05

Entitle by pattern, restrict by policy

> The alternative — entitling capability by capability, per client — produces a permissions matrix nobody can reason about and a sales process that quotes things we cannot deliver.

PATTERN	WHO IT FITS	WHAT IT TURNS ON
Direct enterprise	A merchant contracting with us directly	The full product surface, with autonomy set per decision point and policy authored at merchant level
Platform or PayFac	A party with an estate beneath it	The same surface plus delegation grants, submerchant hierarchy and estate-level policy authority
Embedded	An ISV or software platform embedding payments	A narrower surface with the commerce and reporting obligations met on their behalf
Wholesale	Another PSP or bank buying execution	Execution and resilience without our decisioning — they buy the engine, not the brain

STATE	MEANING
Available	In the pattern, on by default
On activation	In the pattern, off until the client turns it on
Gated	Requires a commercial or risk approval before it can be activated
Policy-restricted	Entitled, but constrained for this transaction by a policy of higher authority — and **reported as restricted, with the authority named**

THE BOUNDARY THAT WILL BE ARGUED ABOUT

Entitlement answers ***has this client bought and been approved for this capability***. Policy answers ***given that they have it, is this particular transaction permitted to use it***.

They have different owners — commercial versus compliance — different change velocities and different override semantics. **Merging them produces a single table in which a sales concession and a regulatory prohibition have equal standing**, and that table will eventually be edited by whoever has a deal closing.

PART

II — Governance

06 One change model

07 Roles and segregation

08 Explainability as an operating requirement

09 The interlock with bank governance

10 Failure modes to design against

One change model

> Everything that alters behaviour goes through the same model. A preference, a strategy parameter, a policy, a plan version and a cutover differ in tier, not in kind.

ONE CHANGE MODEL · THE TIER DECIDES THE APPROVAL, NOT THE TEAM

T1	**Preference**	client self-serve, reversible	automatic
T2	**Strategy parameter**	within declared bounds	mechanical validation
T3	**New strategy binding**	registered, backtested, piloted	owner plus guardrail check
T4	**Policy change**	compiled, simulated, shadowed, canaried	authority-level approval chain
T5	**Plan version**	new execution graph	architecture plus staged exposure
T6	**Emergency prohibition**	restrict only, additive, time-limited	dual control, human, always
T7	**Irreversible**	cutover, retirement, schema break	human by construction — no rollback exists

MACHINES GOVERN MECHANICAL VALIDATION. HUMANS GOVERN BOUNDS, OBJECTIVES, EXCEPTIONS AND ACCOUNTABILITY.

T6 and T7 stay human by construction, because the failure mode of automated approval on an irreversible change has no rollback.

— Modularity does not remove risk; it converts release risk into configuration risk. The tier table is what stops that conversion being a downgrade.

PRINCIPLE	WHAT IT RULES OUT
The tier decides the approval, not the team.	A senior team shipping a T5 change through a T2 path because they are trusted.
Nothing unowned, nothing immortal.	Every artefact has an owner and an expiry. A strategy nobody will defend and nobody will delete is how the portfolio rots.
Machines govern mechanical validation; humans govern bounds, objectives, exceptions and accountability.	Committees rubber-stamping things a compiler could have checked, which is where governance credibility goes to die.
Blast radius is calculated before exposure, not estimated after.	“We did not realise it affected that population.”
Rollback is a pointer change wherever it can be.	A rollback that is itself a deployment, performed under incident conditions by people who have not practised it.

THE ONE QUALIFICATION ON GOVERNANCE-AS-CODE

T6 emergency changes and T7 irreversible changes stay human by construction. The failure mode of automated approval on an irreversible change has no rollback, and a validation suite cannot be asked to reason about whether this is the right moment to retire a legacy path.

Everything below those two should be mechanical wherever it can be. A governance process that spends its committee time on things a compiler could check will not have committee time available for the two tiers that need it. [A]

07

Roles and segregation

ROLE	OWNS	MAY NOT ALSO BE
Author	Writes the strategy, policy or configuration	The approver for the same change
Approver	Accepts the change at its tier	The author, and for T6 there must be two of them
Exposure owner	Decides what share of traffic sees it, and when it ramps	The author
Guardrail owner	Sets the thresholds that abort it	The exposure owner — otherwise the person who wants it live also decides what counts as harm
Holdout owner	Governs the permanent holdout	**Anyone in the commercial line** — see below
Domain owner	Accountable for the artefact's existence at review	Absent. Every artefact has one

THE HOLDOUT OWNER IS THE ONE THAT MATTERS

A permanent holdout is a standing cost: two percent of every population receives the baseline and no optimisation. Someone will eventually observe, correctly, that shrinking it would raise measured performance.

A holdout whose size is renegotiable is a holdout that gets negotiated down. So its size is a platform-level parameter with the same immutability discipline as an emergency prohibition, and its owner sits outside the line that benefits from shrinking it. The same argument applies to the exploration budget, and both are open items — D6 and D4.

Explainability as an operating requirement

> Not a documentation exercise. A measurable property with a budget, because an explanation that takes two days to assemble is not an explanation.

QUESTION	WHO ASKS IT	THE BUDGET
Why was this transaction declined?	Merchant support, constantly	Self-serve, under two minutes
Why did this payment go to that processor?	Enterprise merchants, in QBRs	Self-serve
Why did we *not* use the other one?	Sophisticated merchants, once they know to ask	Self-serve — and it costs nothing to produce, because the considered set is already logged
What rule was in force when this ran, and what did we believe was in force?	Compliance, in a finding	Answerable from the bitemporal record, without a reconstruction project
Which experiment was this transaction in?	Merchant, on request	Queryable
What changed, that made last Tuesday different?	Operations, during an incident	The determinism snapshot, immediately

Explanation coverage is a tracked metric, not a feature. The failure state is a platform that can explain any decision *in principle* and cannot explain this one *today*.

The interlock with bank governance

> Being a bank is the source of the funding advantage and the source of the slowness. They are the same fact, and a plan that claims the first while complaining about the second has not decided what it is.

BANK PROCESS	WHAT IT BINDS	HOW THE PLATFORM MODEL MEETS IT
Model risk management	Any model influencing authorisation, decline or funding: documented development, independent validation, ongoing monitoring, and a defensible account of why it decided what it decided	**This is the binding constraint on learned strategies, not the machine learning.** It also argues for the architecture we have: predictions feeding a governed decision layer, rather than a model that decides
Change management	Production changes, with a tier and an approval	The change model maps directly. The work is proving a strategy promotion is a governed change, not an untracked one
Third-party risk	Every processor, acquirer and data source	Each connector is a third-party relationship with a lifecycle, which is a cost multi-acquiring has to carry
Regulatory change	Rules arriving with a date	Bitemporality was built for this. The emergency channel exists because shadow-then-canary is right for a normal change and useless for a sanctions addition
Audit	Evidence, on demand	The journal, the determinism snapshot and the decision records are the audit surface. They were built for learning and they answer this too

THE CONSTRAINT THAT IS ALSO THE MOAT

A competitor without a charter reaches instant settlement through a partner bank or a capital arrangement, which adds a counterparty, a margin and a ceiling on how far acceleration can be pushed. We do not have that hop. That is a structural advantage that no amount of competitor execution erodes.

The same balance sheet is why stablecoin issuance, merchant-of-record and agent custody run through approval paths a fintech does not have. **Claim the advantage or complain about the constraint, but not both in the same document.** [J]

Failure modes to design against

FAILURE MODE	HOW IT ARRIVES	THE DESIGNED DEFENCE
Configuration becomes the new monolith	Enough policies, strategies and preferences accumulate that no one can predict the effect of a change	Compilation with conflict detection; mandatory expiry on everything; inert reporting; blast radius before exposure
The holdout is negotiated away	A quarter where performance matters and two percent looks expensive	Platform-level parameter, non-commercial owner, and — from CI-001 — make it load-bearing for the company's headline metric
Attribution becomes a compensation instrument	Teams rewarded on attributed basis points	Publish attribution for transparency; **do not make it a performance-management input**. If it becomes one, the attribution model becomes a compensation instrument and its honesty will not survive that
Exceptions become the policy	Each one reasonable, none expiring	Mandatory expiry, no "never"; lapse silently and the policy reasserts; count and age reported to product monthly; owned by product, not by the requester
Thirty-three contexts become thirty-three bottlenecks	Each staffed at one person	Nine product groups. Bounded contexts stay as the artefact boundary
The evidence layer is deferred because nothing depends on it yet	It has no user in quarter one	It is the only irreversible item in the programme. Every month of delay is a month of evidence that cannot be reconstructed
Shadow becomes theatre	A zero-divergence gate that the team meets by tuning the new engine to imitate the old	Two-class gate: invariants at zero always; decisions ***explained***, not identical
The platform optimises approval and loses money	Approval up forty basis points, and cost, fraud and disputes quietly worse	The outcome graph, the economic objective, and guardrails at every horizon rather than the fastest one

PART

III — Getting there

11 The build sequence

12 Six migration stages

13 The first ninety days

14 What it costs

The build sequence

> Ordered by risk-adjusted value, not by capability grouping. The first wave is deliberately unglamorous, and it is the wave that unblocks everything after it.

WAVE	WHAT SHIPS	WHY HERE
Wave 0 — evidence	Propensity, considered set and fallback logged from the ***current*** engine. The permanent holdout reserved. The controlled segment vocabulary. The capability matrix verified against processors.	**No behaviour change, and none of it can be backfilled.** Everything in waves 2 and 3 depends on data that only exists if this starts now
Wave 1 — deterministic value	Least-cost debit, PINless routing, interchange qualification, Level 2 and 3 submission, message model selection, circuit breakers and failover	Real value in weeks, and most of it can run on the legacy engine. Levels one and two of the maturity model carry the majority of realisable value
Wave 2 — the platform	Credential vault and Forward API on new writes; the resource model as a facade; instrument enrichment; journal ingestion; the control plane in advisory mode	Observability before control. The platform becomes real without owning a write
Wave 3 — parametric	Health weighting, latency awareness, amount-band tuning, decline-code-aware recovery, the decision platform registry and first controlled comparisons	Reviewable by inspection, so governance is tractable while the evidence base fills
Wave 4 — learned	Approval prediction and issuer affinity, mid-market first, on pooled cohorts	Gated on the evidence test ***and*** model-risk validation, not on a date
Wave 5 — joint	Credential and route scored as a tuple; dunning optimisation	Requires shared attribution to be accepted, which is a conversation before it is a build

THE ORDERING RULE

Prefer what is **cheap now and expensive later** over what is **expensive now and cheaper later**. Evidence logging, contract schema and rail-neutral abstractions belong in the first category; models, agent wallets and new rails belong in the second.

That rule is developed in CI-001 section 19, and it is the single most useful thing to carry into a prioritisation argument, because it converts a debate about ambition into a question about cost curves.

Six migration stages

> Observe, shadow, pilot, expand, default, retire. Shadow is the one that matters, and it is the one most often skipped because it produces no merchant-visible benefit.

STAGE	WHAT IS TRUE	GATE TO THE NEXT
Observe	Legacy runs. We log decisions, propensity and fallbacks from it. Nothing changes for anyone.	Coverage across the eligible population, and the records validate
Shadow	The new path executes alongside and dispatches nothing. Decisions are diffed against legacy outcomes.	**Invariants at zero, and every decision divergence explained by category.** Not zero divergence — the new engine is meant to decide differently
Pilot	A named, informed, opt-in population runs live on the new path.	Acceptance non-inferior, latency within budget, and a full settlement cycle reconciled
Expand	The ramp ladder, step by step, with soak periods.	Each step's own criteria. Strategic accounts are never in an early step
Default	New volume arrives on the new path by default; legacy is the exception.	Sustained, with the rollback path still exercised
Retire	The legacy path is disabled for the slice.	— and this one is irreversible, which is why it is a T7 change

WHY SHADOW IS THE STAGE THAT GETS CUT, AND WHY IT SHOULD NOT BE

Shadow costs real infrastructure and delivers nothing a merchant can see. Every pressure points at shortening it.

But shadow is the only stage where a disagreement between old and new is free. After it, every divergence is either a merchant-visible defect or an unnoticed one — and the unnoticed ones are discovered at settlement, which is the expensive place to discover anything.

The second reason: the shadow diff is what produces the ***categories*** of intended difference. Without that catalogue, nobody can distinguish a working improvement from a bug, and the team ends up tuning the new engine to imitate the old one just to make the dashboard green. [J]

The first ninety days

> What actually happens, and what does not need a funding decision to start.

WINDOW	DELIVERABLE	OWNER	THE OUTCOME THAT PROVES IT
0–30	Propensity, considered set and fallback emitted from the current routing and retry decisions	Routing engineering	A day of production decisions is queryable with its counterfactual attached
0–30	**Permanent holdout reserved**	Data science	Two percent of every population is excluded from experiments and exploration, with a named non-commercial owner
0–30	Decision point registry, with terminal fallbacks and experimentable flags	Architecture	Five decision points registered; a sixth could be added by registration alone
0–30	Controlled segment vocabulary and the allocation ledger	Architecture	A second experiment on an occupied layer is rejected at registration
0–30	Capability matrix, verified against processors	Product	Every capability claim has a processor confirmation behind it, not an assertion
30–60	The seventeen definitions, confirmed and published	Named per row	Two teams quoting the same metric mean the same thing
30–60	Decision contract schema, enforcing the three mandatory fields at registration	Architecture	A decision point that cannot emit propensity fails registration
30–60	First deterministic strategies registered and backtested	Routing product	A strategy deploys without a release
30–60	Reason code vocabulary published with merchant text	Developer platform	A merchant-visible decline has canonical text and a stated action
60–90	First controlled comparison, on a deterministic strategy	Product and data science	One readout with a denominator named, fidelity reported and an interval
60–90	Guardrails wired, with automated abort	Data science	An arm aborts without a human in the loop, and the abort is the action rather than the ticket
60–90	Economic readout and the baseline ratchet	Product and finance	A promotion recompiles the baseline, and the next comparison is against the new one

THE 90-DAY SUCCESS CRITERION, STATED AS ONE SENTENCE

At day ninety we can state the effect of one payment decision on approval rate, with a confidence interval, against a concurrent control, on production traffic — and the number could have come out negative.

That is the whole capability. Everything else in the programme is scale on top of it, and nothing else in the first ninety days is worth having if that sentence is not true at the end of them.

Note what is not in it: no merchant-visible change, no funding decision, no production rollout, no new platform. The work is establishing an operating model and collecting evidence that cannot be created retroactively. [A]

What it costs

> The plan above is roughly three squads for a quarter. Saying so is more useful than an optimistic silence that becomes a slipped date.

WORKSTREAM	SHAPE	NOTE
Evidence logging on the legacy engine	One squad, most of the quarter	The awkward part is the legacy engine, not the logging
Registry, contract schema, allocation ledger	One squad	Mostly greenfield, which is why it looks faster than it is
Definitions, vocabularies, capability matrix	Not a squad — calendar time and meetings	A week of conversations spread over six weeks, and it blocks more than it looks like it does
Guardrails, readout and the ratchet	Half a squad, back-loaded	Depends on the first two
Critical-path only	**Roughly 2.7 squads**	[C] — from the delivery backlog, at a nominal velocity
Full 90-day scope	**Roughly 3.1 squads**	[C] — the difference is the items that could slip to day 120 without breaking anything downstream

THE HONEST VERSION OF THE ASK

Nothing here requires a new platform, a new budget line or a merchant-visible change. It requires roughly three squads for a quarter, and two decisions that cost nothing to take and are expensive to defer: reserve the holdout, and name the owner of the exploration budget.

If only one thing is approved, it should be the holdout, because it is the only item on the list whose option expires. Everything else can start in month four at a cost of three months. The holdout cannot — three months of unreserved holdout is three months of causal evidence that will never exist.

ST-001 v1.0 · strategy and execution · 27 September 2026. Consolidates the governance and operating model, the execution half of PV-2027 and the delivery backlog. Companion to VS-001. Implements EA-001, PD-001 and PD-002. Open items are tracked in DOC-000 section 06. [A] actual · [C] calculated · [J] judgement.